Ema Lovrić, 11.5.2026.

Assignment 5: High availability with load balancing

https://github.com/elovric13/eutech.git

1. Set up two web servers in containers

Two Nginx web servers were set up using Docker containers. Each server was built from a custom Dockerfile based on the nginx:alpine image, with a unique index.html file. Docker and Docker Compose were used to manage the containers.

```
ubuntu26@ubuntu26:~/load-balancer$ cat web1/Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
ubuntu26@ubuntu26:~/load-balancer$ cat web2/Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Figure 1: web1/Dockerfile and web2/Dockerfile

```
ubuntu26@ubuntu26:~/load-balancer$ cat web1/index.html
<!DOCTYPE html>
<html>
<body>

<h1>First web server</h1>
<p>This is the FIRST web server</p>

</body>
</html>
ubuntu26@ubuntu26:~/load-balancer$ cat web2/index.html
<!DOCTYPE html>
<html>
<body>

<h1>Second web server</h1>
<p>This is the SECOND web server</p>

</body>
</html>
```

Figure 2: web1/index.html and web2/index.html

2. Install and configure a load balancer with HAProxy

HAProxy was configured as the load balancer using official Docker image. A configuration file haproxy.cfg was created and mounted into the HAProxy container as a read-only volume. HAProxy listens on port 80 and forwards incoming requests to the two backend Nginx servers.

```
ubuntu26@ubuntu26:~/load-balancer$ cat haproxy/haproxy.cfg
global
    daemon

defaults
    mode http
    timeout connect 5s
    timeout client  30s
    timeout server  30s

frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    server web1 web1:80 check
    server web2 web2:80 check
```

Figure 3: haproxy.cfg configuration file

3. Configure round-robin to distribute traffic between the servers

Round-robin load balancing was configured in the HAProxy backend section. The health check flag was added to both server entries so HAProxy can detect and respond to server failures automatically.

```
ubuntu26@ubuntu26:~/load-balancer$ cat docker-compose.yml
version: '3'

services:
  web1:
    container_name: web1
    build: ./web1
    networks:
      - webnet

  web2:
    container_name: web2
    build: ./web2
    networks:
      - webnet

  haproxy:
    image: haproxy:latest
    container_name: haproxy
    ports:
      - 80:80
    volumes:
      - ./haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    depends_on:
      - web1
      - web2
    networks:
      - webnet

networks:
  webnet:
    driver: bridge
```

Figure 4: docker-compose.yml

```
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>First web server</h1>
<p>This is the FIRST web server</p>

</body>
</html>
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>Second web server</h1>
<p>This is the SECOND web server</p>

</body>
</html>
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>First web server</h1>
<p>This is the FIRST web server</p>

</body>
</html>
```

Figure 5: curl results showing round-robin alternation

4. Simulate failure of one web server and verify that the load balancer redirects traffic correctly

To simulate a server failure, web1 was stopped using the docker stop web1 command. All traffic was automatically redirected to web2 with no errors returned to the client.

After running docker start web1, round-robin distribution resumed automatically between both servers without any manual reconfiguration.

```
ubuntu26@ubuntu26:~$ sudo docker stop web1
web1
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>Second web server</h1>
<p>This is the SECOND web server</p>

</body>
</html>
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>Second web server</h1>
<p>This is the SECOND web server</p>

</body>
</html>
ubuntu26@ubuntu26:~$ curl http://localhost
<!DOCTYPE html>
<html>
<body>

<h1>Second web server</h1>
<p>This is the SECOND web server</p>

</body>
</html>
```

Figure 6: docker stop web1 and curl results showing only web2 responding

5. Benefits of load balancing in enterprise environments

High Availability: Traffic is automatically rerouted if a server fails, eliminating single points of failure

Scalability: New servers can be added to the backend pool without downtime or configuration changes to clients

Fault Tolerance: Health checks detect unhealthy servers and remove them from rotation automatically

Performance: Distributing requests across multiple servers reduces response times under heavy load

Zero Downtime Deployments: Servers can be updated one at a time while others continue serving traffic

Centralized Traffic Management: A single entry point simplifies logging and access control

Cost Efficiency: Horizontal scaling is more cost-effective than vertical scaling